

PYTHON MODULE 01

Sure! Here's a simplified yet slightly detailed, exam-ready explanation of **Module 1: Advanced Functions** with all subtopics properly covered.

Module 1: Advanced Functions

This module is about *Functional Programming* in Python. Functional Programming means writing code where **functions are treated like variables**, and they are used to process data in a clean and reusable way.

1. Introduction to Functional Programming

Functional Programming is **a programming style that focuses on using pure functions** (functions without side effects) and **avoids changing data or state**.

Key points:

- Functions are **first-class citizens**: They can be passed as arguments, returned from other functions, and assigned to variables.
 - It uses **immutable data** (data that can't be changed).
 - Examples of functional programming features in Python include: **map(), filter(), reduce(), lambda, etc.**
-

2. Functions as Objects

In Python, **functions are objects**. This means:

- You can **assign a function to a variable**.
- You **can pass a function as an argument** to another function.
- You can return a function from another function.

Example:

```
def greet(name):  
    return f'Hello {name}'
```

```
say_hello = greet # assigning function to a variable  
print(say_hello("Conan"))
```

3. Lambda Functions

Lambda functions are small anonymous functions written in a single line.

Syntax:

lambda arguments: expression

Example:

```
add = lambda x, y: x + y  
print(add(3, 4)) # Output: 7
```

Used when you need a function for a short period (like inside `map()` or `filter()`).

4. List Comprehensions

A short way to create lists using a single line of code.

Example:

```
squares = [x*x for x in range(5)]
```

```
# Output: [0, 1, 4, 9, 16]
```

It can also include conditions:

```
evens = [x for x in range(10) if x % 2 == 0]
```

```
# Output: [0, 2, 4, 6, 8]
```

high order functions



5. List Comprehensions with Sequence Processing

We often use **map**, **filter**, and **reduce** along with list comprehensions or instead of loops.

a. **map(function, iterable)**

Applies a function to every item in an iterable.

```
nums = [1, 2, 3]
```

```
squared = list(map(lambda x: x**2, nums))
```

```
# Output: [1, 4, 9]
```

b. **filter(function, iterable)**

Filters elements based on a condition.

```
even = list(filter(lambda x: x % 2 == 0, nums))
```

```
# Output: [2]
```

c. `reduce(function, iterable)`

Reduces the list to a single value.

```
from functools import reduce
```

```
sum_all = reduce(lambda x, y: x + y, nums)
```

```
# Output: 6
```

6. Iterators

Iterators are objects you can loop over. They follow the **Iterator Protocol**, which includes `__iter__()` and `__next__()` methods.

Example:

```
nums = iter([1, 2, 3])
```

```
print(next(nums)) # Output: 1
```

```
print(next(nums)) # Output: 2
```

7. Generators

Generators are functions that return an iterator using the `yield` keyword. They are memory-efficient and used for large data.

Example:

```
def count_up_to(n):
```

```
    i = 1
```

```
    while i <= n:
```

```
yield i
```

```
i += 1
```

```
for num in count_up_to(3):
```

```
    print(num)
```

```
# Output: 1 2 3
```

8. Recursion

Recursion is when a function calls itself to solve a problem in smaller parts.

Example (factorial):

```
def fact(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    return n * fact(n-1)
```

```
print(fact(5)) # Output: 120
```

Make sure to define a **base case** to avoid infinite loops.

9. Closures

A closure is created when a **nested function remembers the variables of its outer function**, even after the outer function is done executing.

Example:

```
def outer(x):  
    def inner(y):  
        return x + y  
    return inner
```

```
add5 = outer(5)
```

```
print(add5(3)) # Output: 8
```

10. Function Annotators (Type Hints)

These are optional hints to tell what type of input a function expects and what it returns.

Example:

```
def add(x: int, y: int) -> int:  
    return x + y
```

They **do not enforce the type**, but help in readability and tools like IDEs.

11. Decorators

A decorator **is a function that modifies another function without changing its code**. It is often used for logging, authentication, etc.

Example:

```
def decorator(func):  
    def wrapper():  
        print("Before function call")  
        func()  
        print("After function call")  
    return wrapper
```

@decorator

```
def say_hi():  
    print("Hi!")
```

```
say_hi()
```

Summary Table

Concept	Key Idea
Functional Programming	Treat functions like variables
Lambda	One-line anonymous function

Concept	Key Idea
List Comprehension	Short way to create lists
Map	Apply function to each element
Filter	Keep only elements that match a condition
Reduce	Combine all elements into one
Iterators	Object with <code>__next__()</code>
Generators	Iterator using <code>yield</code> , memory efficient
Recursion	Function calling itself
Closures	Function remembers outer variable
Type Hints	Show expected input/output types
Decorators	Add extra behavior to functions

Let me know if you'd like a PDF version or if we should do the next module too! 😊